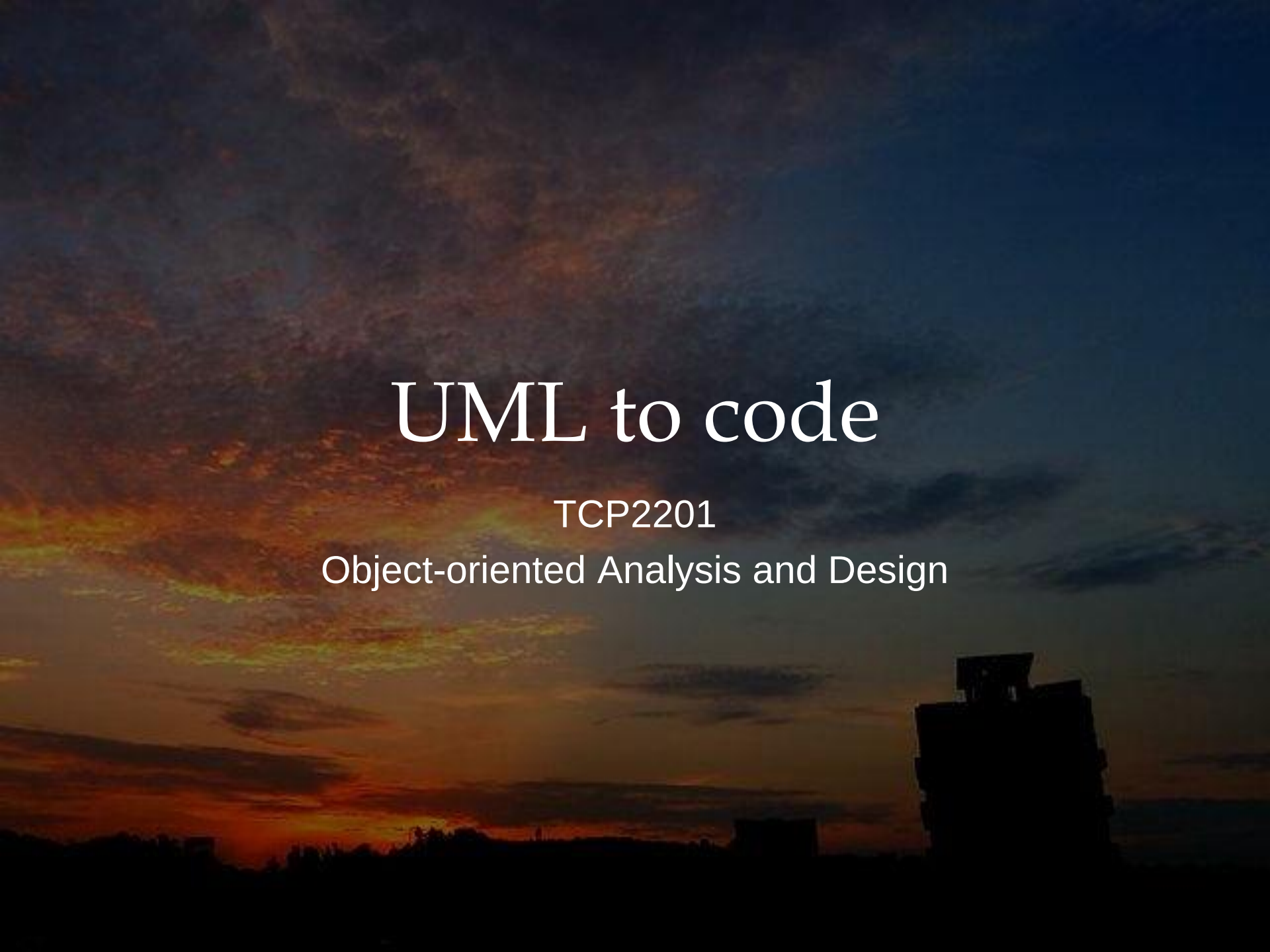




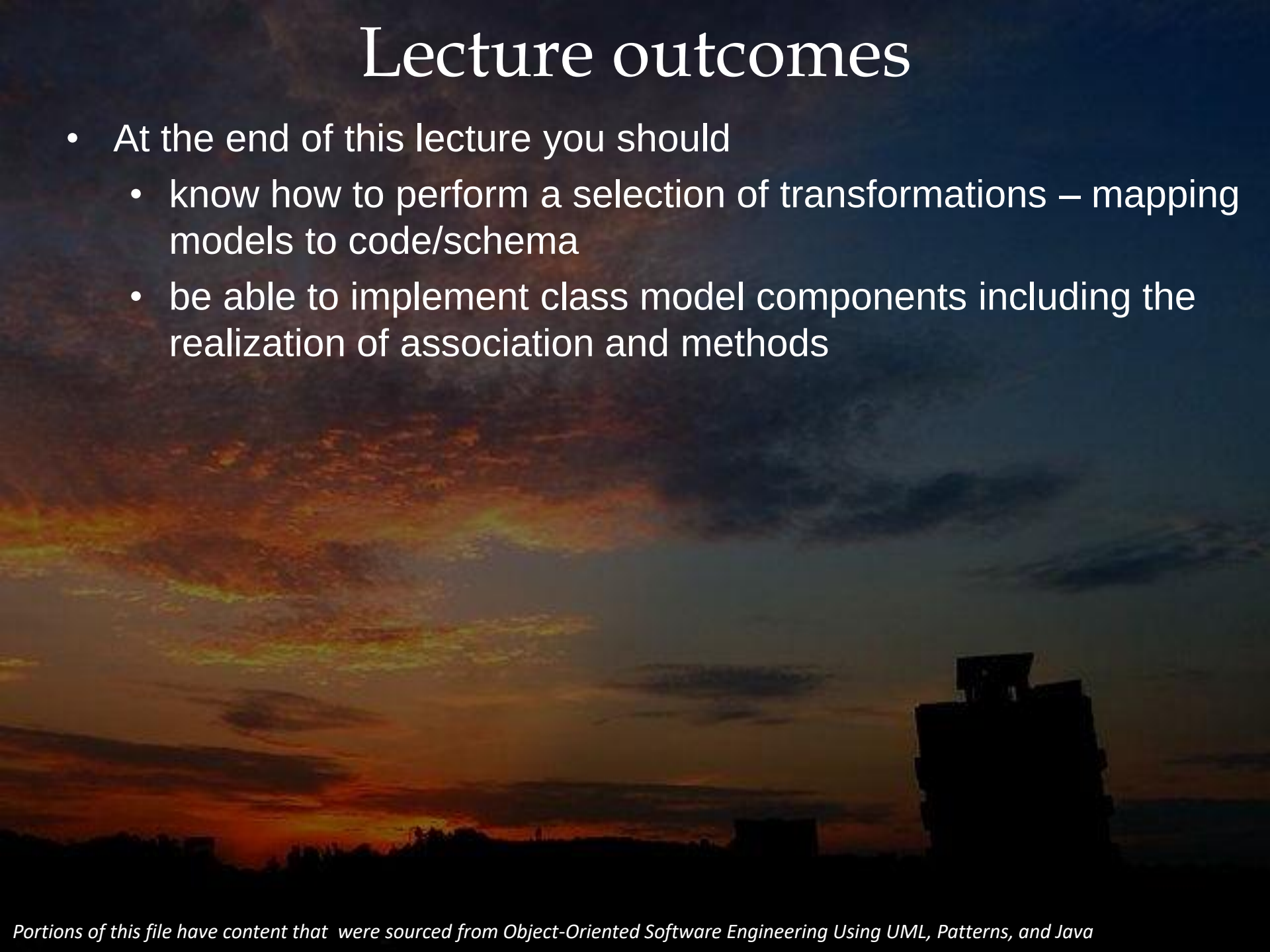
UML to code

TCP2201

Object-oriented Analysis and Design

Lecture outcomes

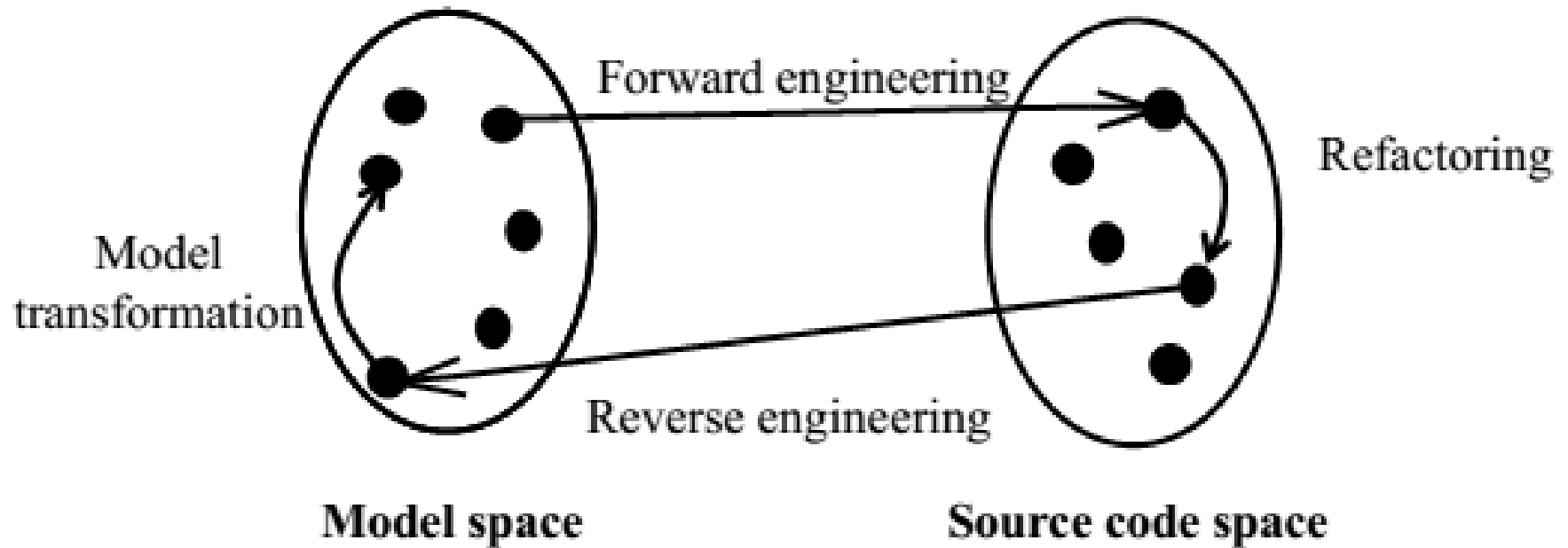
- At the end of this lecture you should
 - know how to perform a selection of transformations – mapping models to code/schema
 - be able to implement class model components including the realization of association and methods



Mapping activities

- The task of performing transformation on the object model
- *Problem?*
Programming languages do not support the concept of association the way it is presented visually in the model → requires transform of associations into collections of object references
- Example:
 - If programming language does not support contracts – you write code to detect and handle contract violations
 - Revise and update interface specifications of objects to accommodate new requirements from client
- The transform process is intellectually non-challenging but are repetitive and prone to error due to repetition

Typical transformations



Additional terminology

- Not shown in the diagram on previous slide are other possible transformations
 - Roundtrip engineering
 - Forward+reverse engineering
 - Re-engineering
 - Used in context of project management
 - Process of extracting requirements / design from existing systems or legacy code without any available design documents

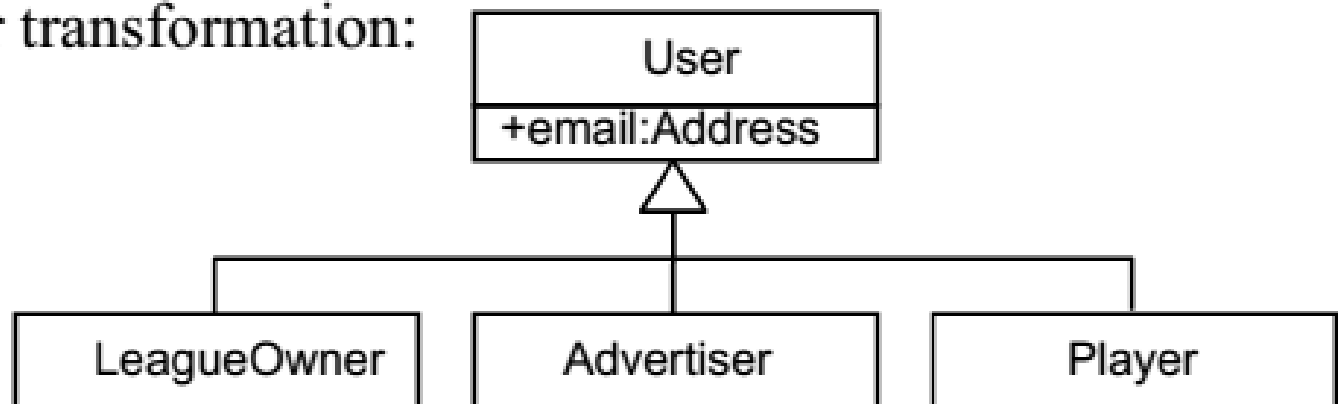
Model transformation

- Model transformations involve processes to translate between different types of models or to correct/update/simplify existing models.
- E.g.

Object design model before transformation



Object design model
after transformation:



Code refactoring

- Code refactoring is the process of restructuring existing computer code – *changing the factoring* – without changing its external behavior
- Usually done for reasons of improved readability, reduced complexity, improved maintainability and improved extensibility.

```
public class Player {  
    private String email;  
    //...  
}  
  
public class LeagueOwner {  
    private String eMail;  
    //...  
}  
  
public class Advertiser {  
    private String email_address;  
    //...  
}
```

```
public class User {  
    protected String email;  
}  
  
public class Player extends User {  
    //...  
}  
  
public class LeagueOwner extends  
    User {  
    //...  
}  
  
public class Advertiser extends  
    User {  
    //...  
}
```

Forward engineering

- Process of transforming models to code

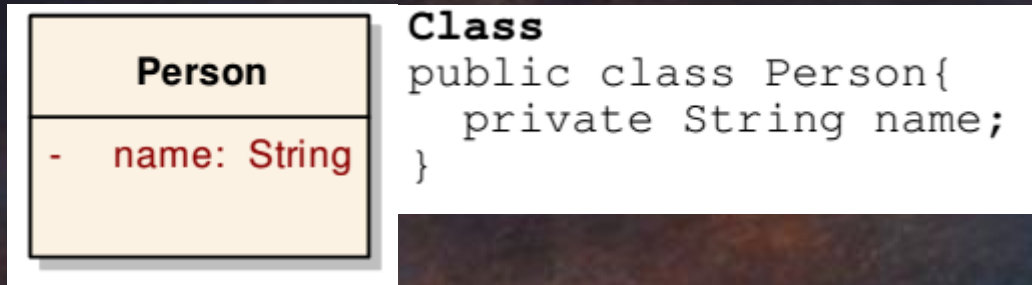


```
public class User {
    private String email;
    public String getEmail() {
        return email;
    }
    public void setEmail(String value) {
        email = value;
    }
    public void notify(String msg) {
        // ....
    }
    /* Other methods omitted */
}
```

```
public class LeagueOwner extends User {
    private int maxNumLeagues;
    public int getMaxNumLeagues() {
        return maxNumLeagues;
    }
    public void setMaxNumLeagues
        (int value) {
        maxNumLeagues = value;
    }
    /* Other methods omitted */
}
```

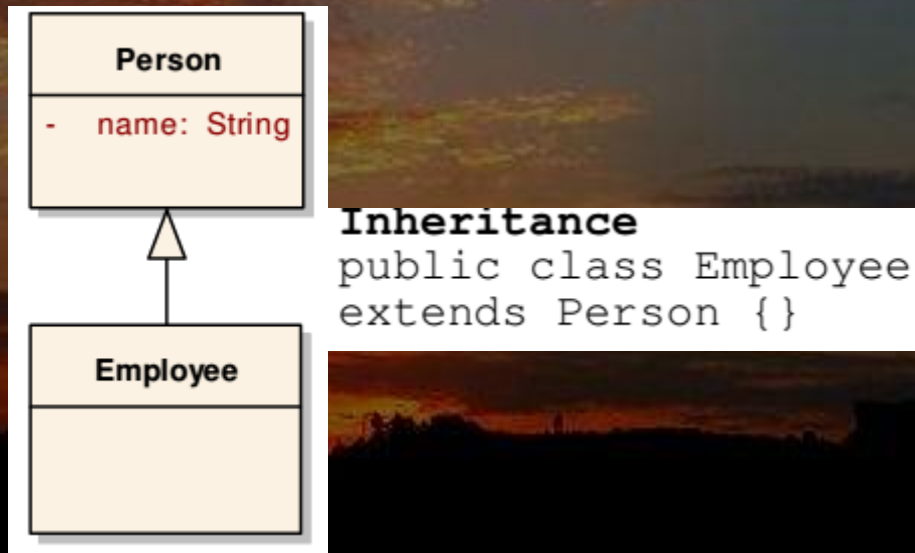
- Obviously the opposite is true for reverse engineering

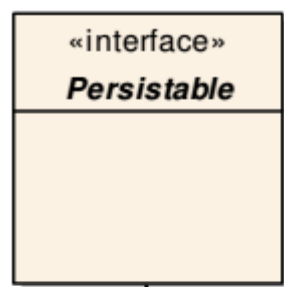
Typical UML→Java transforms



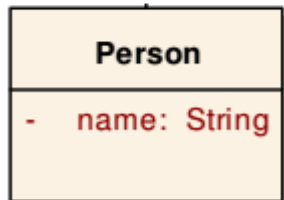
Association

```
public class Person{  
    private String name;  
    private Address address;  
}  
  
public class Address{}
```





«realize»

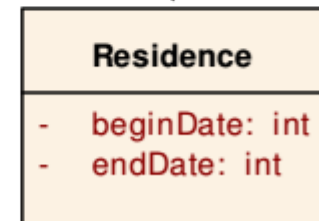
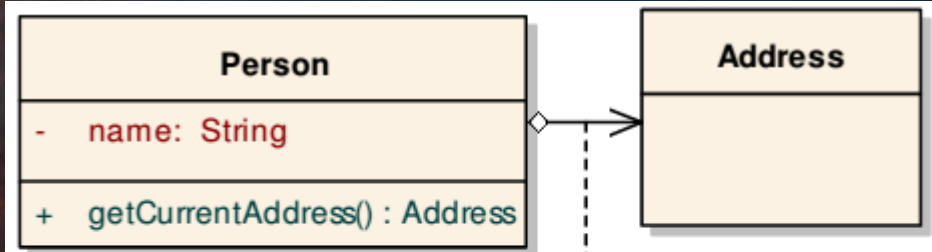


```

public interface Persistable{
    public void Save();
}
  
```

```

public class Person
implements Persistable{
    public void Save(){}
}
  
```



Association class

```

public class Person{
    private Residence[] residences;
    public Address getCurrentAddress(){}
}
  
```

```

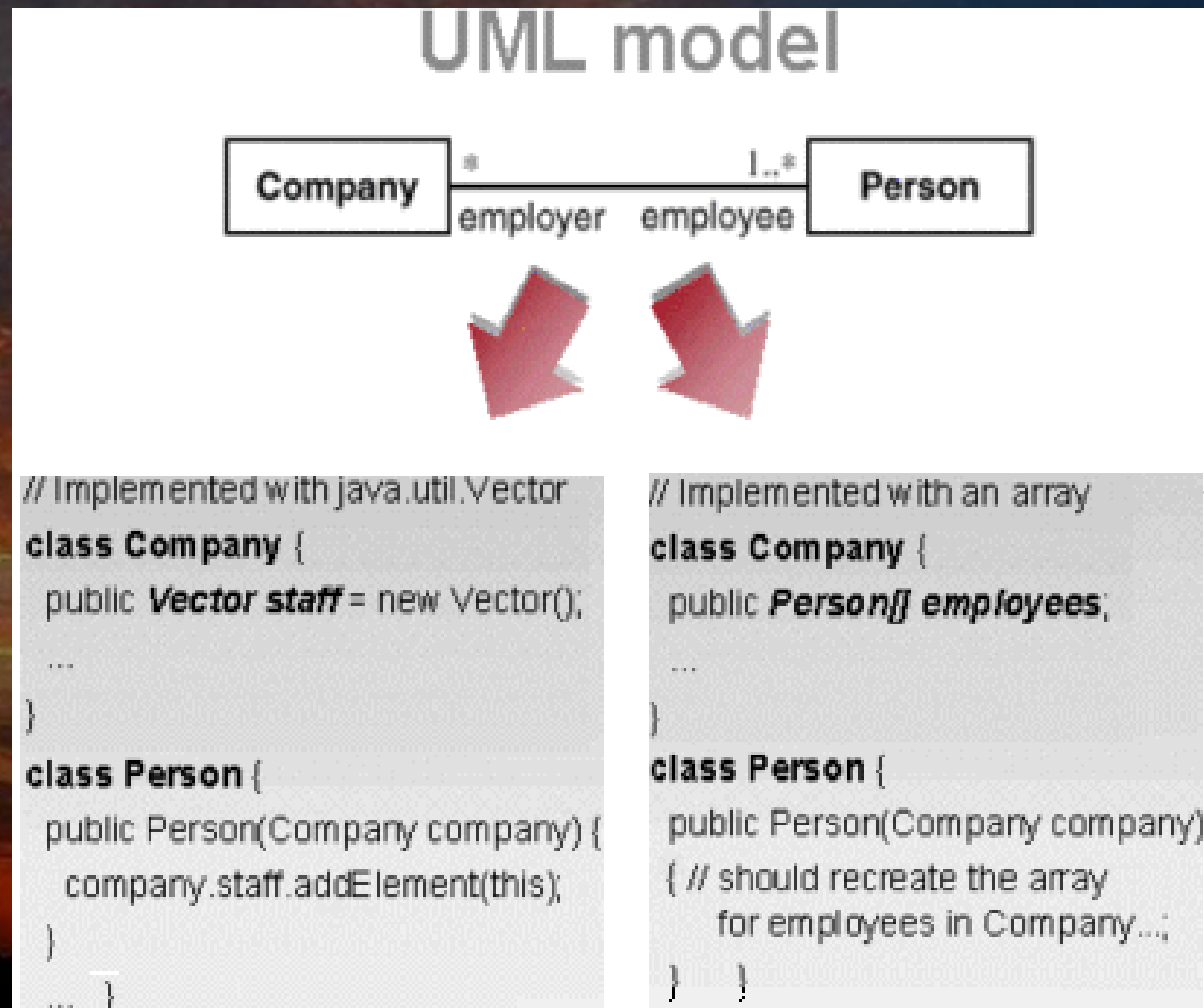
public class Residence{
    private Address address;
    private Date beginDate;
    private Date endDate;
}
  
```

```

public class Address{
}
  
```

UML→Java transforms

- There is no fixed rule to performing transforms – one design/model may lead to one/more code implementations



Other activities

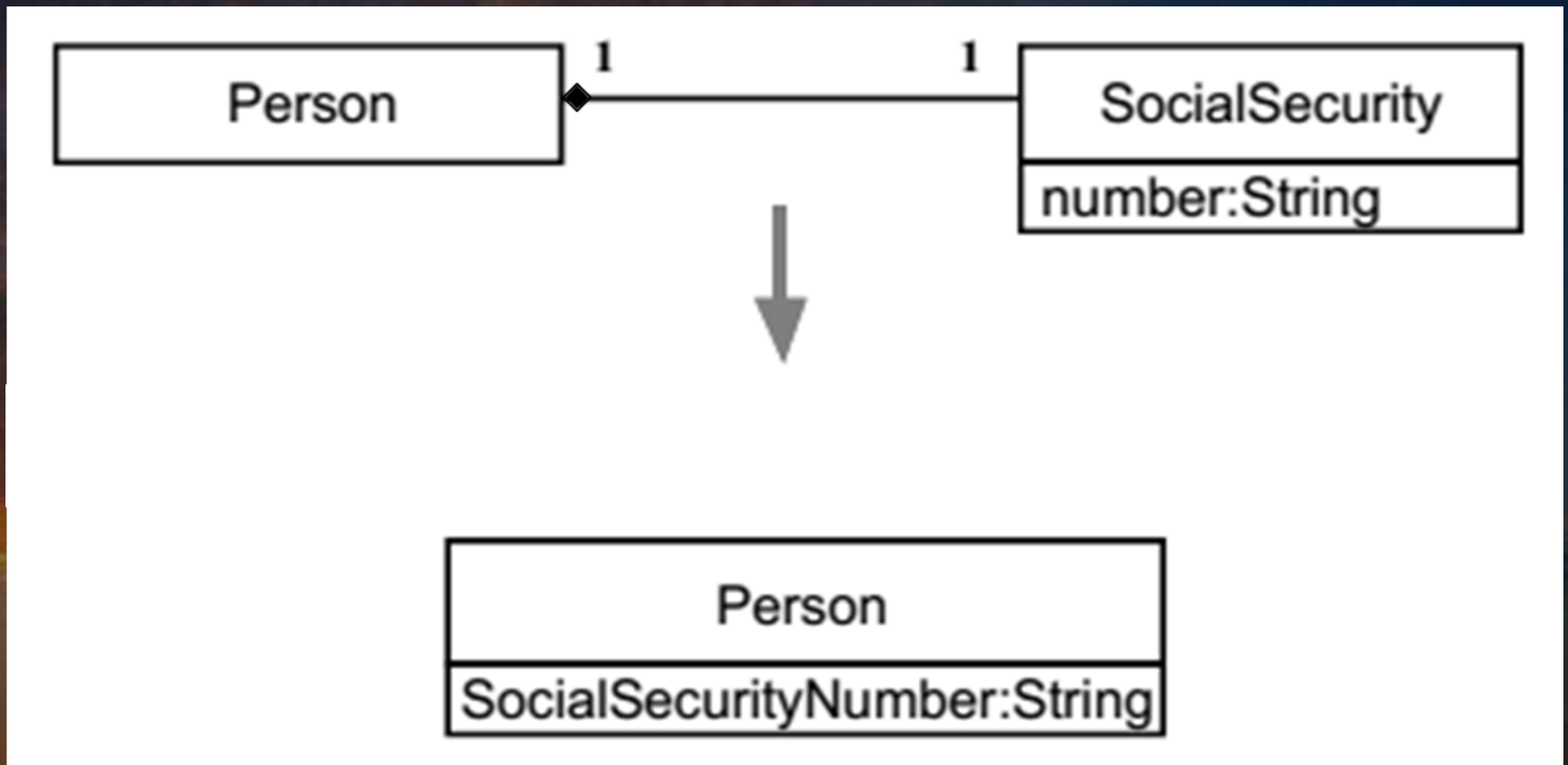
- During the transformation process, other activities take place in parallel to improve the overall design.
- These activities include
 - optimizing the object design model – i.e. restructuring model
 - realizing associations
 - mapping contracts to exceptions
 - mapping objects to database tables



Restructuring activities

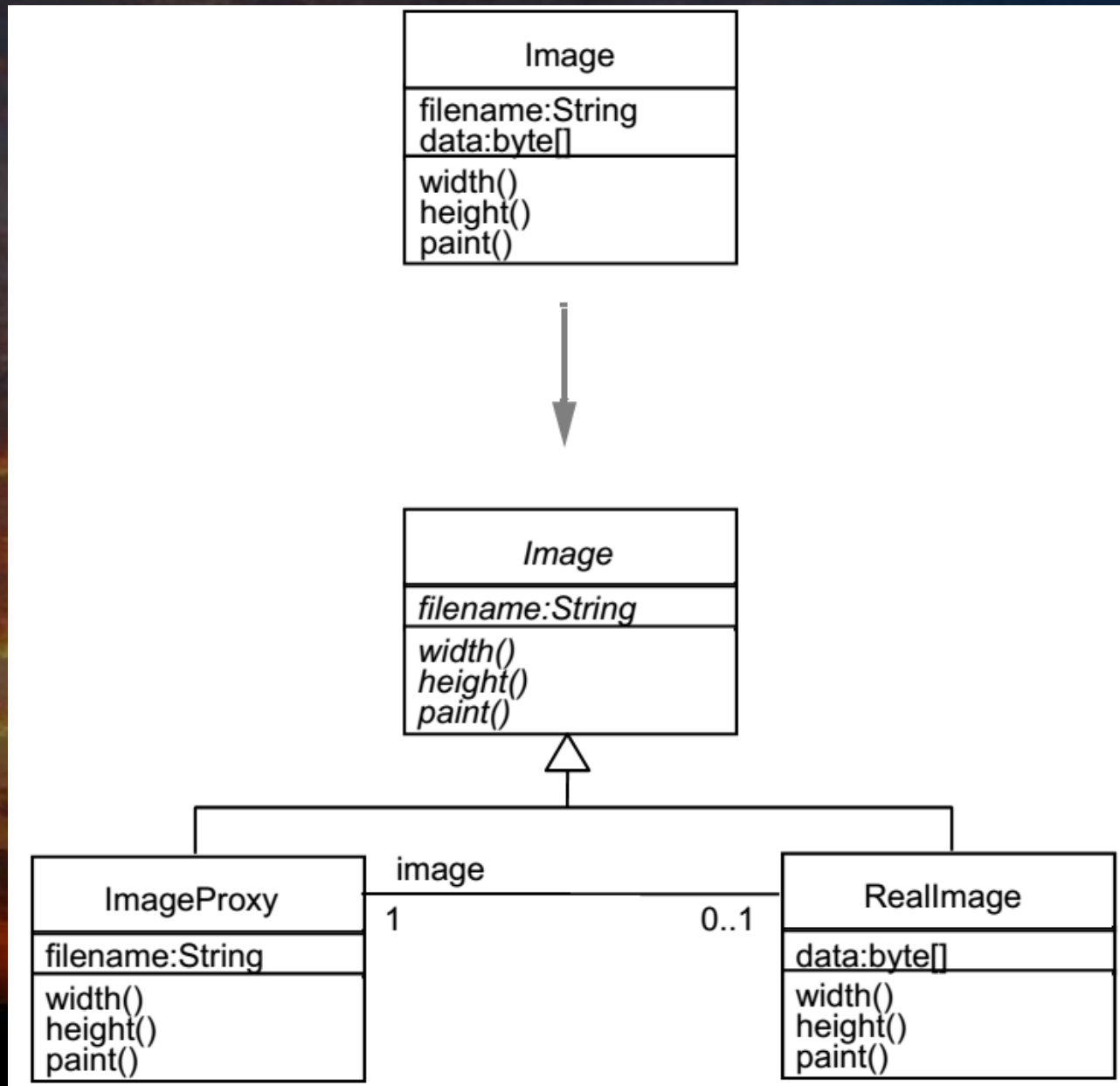
- Transforms the object design model to address performance criteria such as response time or memory utilization
- Requires analysing and revisiting model to locate improvements.
E.g.
 - revisiting inheritance to increase reuse → will promote refactoring in code later
 - revising inheritance to remove dependencies → will promote reuse and better maintainability in objects
- Optimization activities during object design:
 - Add redundant associations to minimize access cost
 - Rearrange computations for greater efficiency
 - Store derived attributes to save computation time
- As an object designer you must strike a balance between efficiency and clarity → Optimizations will make your models more obscure

Restructuring - Collapsing



- Collapse a class into an attribute if the only operations defined on the attributes are getters and setters

Restructuring - delaying



This is the
Proxy
design
pattern

Restructuring – increasing inheritance

- Process to rearrange and adjust classes and operations to prepare for/encourage inheritance
- Abstract common behavior out of groups of classes
 - If a set of operations or attributes are repeated in 2 classes, the classes might be special instances of a more general class.
- Be prepared to change a subsystem (collection of classes) into a superclass in an inheritance hierarchy



Restructuring – building super classes

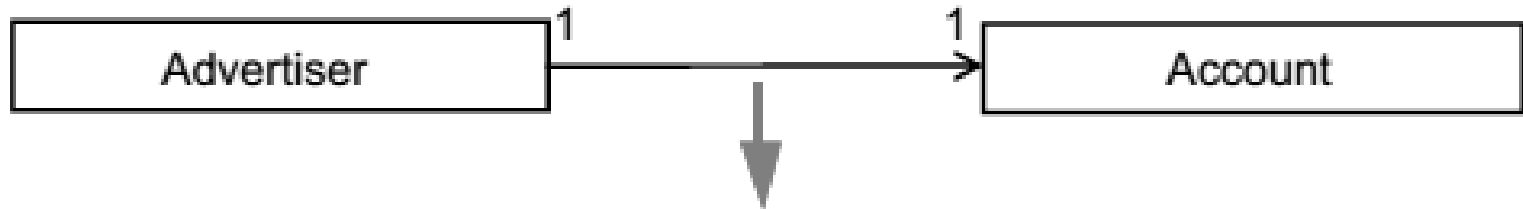
- Prepare for inheritance. All operations must have the same signature but often the signatures do not match:
 - Some operations have fewer arguments than others: Use overloading (Possible in Java)
 - Similar attributes in the classes have different names: Rename attribute and change all the operations.
 - Operations defined in one class but not in the other: Use template methods and overriding.
- Abstract out the common behavior (set of operations with same signature) and create a super class out of it.
- Super classes are desirable. They increase modularity, extensibility and reusability
- Turn the superclass into an abstract interface if possible

Realizing associations

- Strategy for implementing associations
 - Be as uniform as possible
 - Individual decision for each association
- Example of uniform implementations
 - 1-to-1 association:
 - Role names are treated like attributes in the classes and translate to references
 - 1-to-many association:
 - "Ordered many" : Translate to List
 - "Unordered many" : Translate to Set
 - Qualified association:
 - Can translate into to Hash table

Unidirection 1:1 association

Object design model before transformation



Source code after transformation ?

```
public class Advertiser {  
    private Account account;  
    public Advertiser() {  
        account = new Account();  
    }  
    public Account getAccount() {  
        return account;  
    }  
}
```

Bidirectional 1:1 association

Object design model before transformation



Source code after transformation

```
public class Advertiser {  
    /* The account field is initialized  
    * in the constructor and never  
    * modified. */  
    private Account account;  
  
    public Advertiser() {  
        account = new Account(this);  
    }  
    public Account getAccount() {  
        return account;  
    }  
}
```

```
public class Account {  
    /* The owner field is initialized  
    * during the constructor and  
    * never modified. */  
    private Advertiser owner;  
  
    public Account(owner:Advertiser) {  
        this.owner = owner;  
    }  
    public Advertiser getOwner() {  
        return owner;  
    }  
}
```


Bidirectional 1:M association

Object design model before transformation



Source code after transformation

```
public class Advertiser {
    private Set accounts;
    public Advertiser() {
        accounts = new HashSet();
    }
    public void addAccount(Account a) {
        accounts.add(a);
        a.setOwner(this);
    }
    public void removeAccount(Account a)
    {
        accounts.remove(a);
        a.setOwner(null);
    }
}
```

```
public class Account {
    private Advertiser owner;
    public void setOwner(Advertiser newOwner)
    {
        if (owner != newOwner) {
            Advertiser old = owner;
            owner = newOwner;
            if (newOwner != null)
                newOwner.addAccount(this);
            if (oldOwner == null)
                old.removeAccount(this);
        }
    }
}
```

Bidirectional M:N association

Object design model before transformation

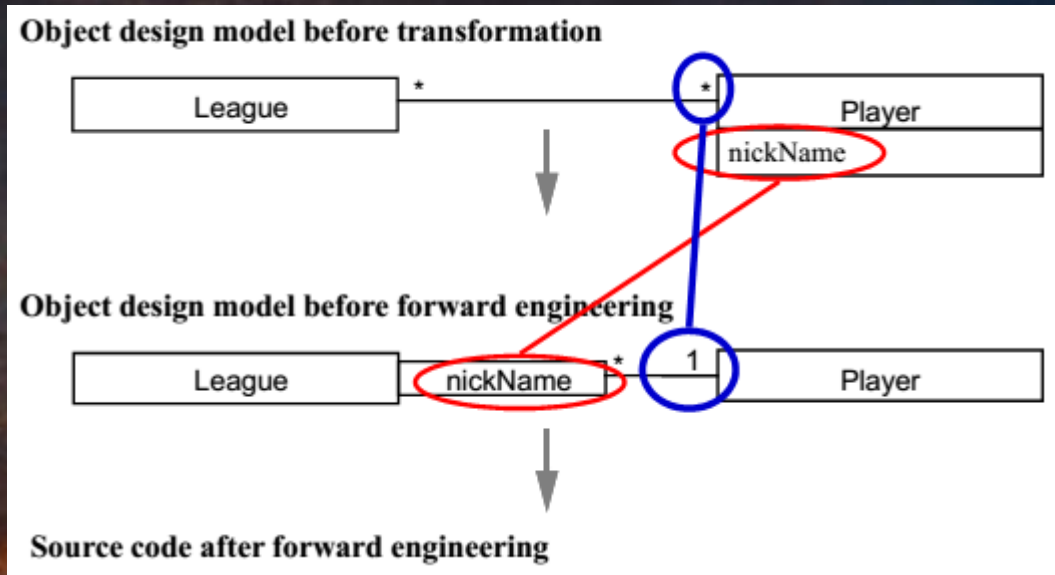


Source code after transformation

```
public class Tournament {  
    private List players;  
    public Tournament() {  
        players = new ArrayList();  
    }  
    public void addPlayer(Player p) {  
        if (!players.contains(p)) {  
            players.add(p);  
            p.addTournament(this);  
        }  
    }  
}
```

```
public class Player {  
    private List tournaments;  
    public Player() {  
        tournaments = new ArrayList();  
    }  
    public void addTournament(Tournament t) {  
        if (!tournaments.contains(t)) {  
            tournaments.add(t);  
            t.addPlayer(this);  
        }  
    }  
}
```

Bidirectional qualified association



```
public class League {
    private Map players;

    public void addPlayer
        (String nickName, Player p) {
        if (!players.containsKey(nickName)) {
            players.put(nickName, p);
            p.addLeague(nickName, this);
        }
    }
}
```

```
public class Player {
    private Map leagues;

    public void addLeague
        (String nickName, League l) {
        if (!leagues.containsKey(l)) {
            leagues.put(l, nickName);
            l.addPlayer(nickName, this);
        }
    }
}
```

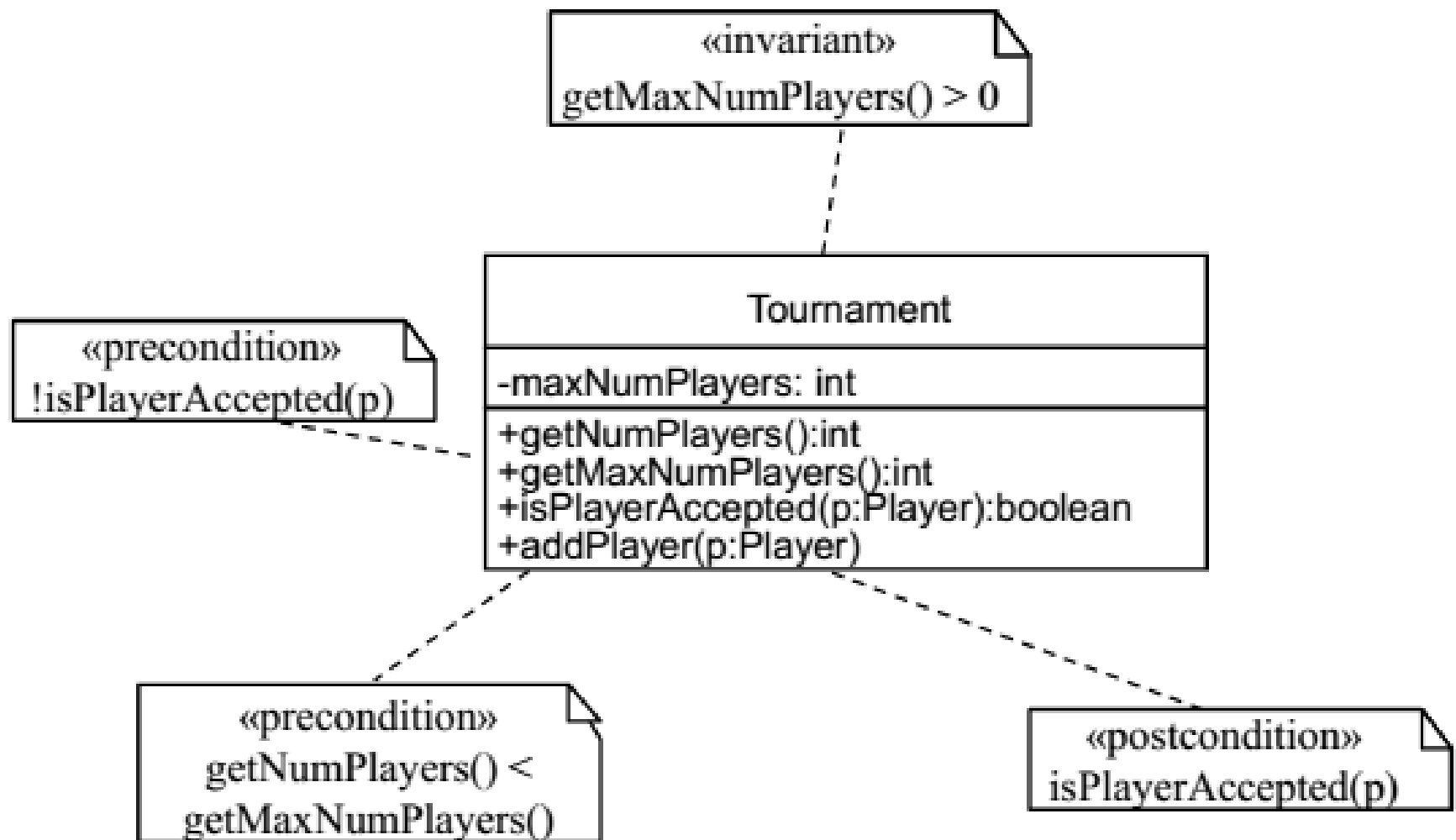
Mapping contracts

- Contracts are restrictions/conditions that are noted in a model
- Most OO languages do not include built-in support for contracts
- However we can use exception mechanisms as building blocks to signal and handle contract violations
 - Java would use the try-throw-catch mechanism



Transforming a contract

- For each operation in the contract, do the following
 - Check precondition:
Check the precondition before the beginning of the method with a test that raises an exception if the precondition is false.
 - Check postcondition:
Check the postcondition at the end of the method and raise an exception if the contract end of the method and raise an exception if the contract is violated.
If more than one postcondition is not satisfied, raise an exception only for the first violation.
 - Check invariant:
Check invariants at the same time as postconditions.
 - Deal with inheritance:
Encapsulate the checking code for preconditions and postconditions into separate methods that can be called from subclasses



```

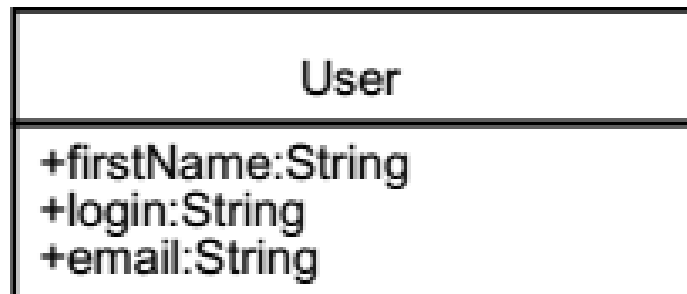
public void addPlayer(Player p) throws KnownPlayerException {
    boolean exceptionThrown = false;
    try {
        // pre conditions
        if (tournament.isPlayerAccepted(p)) {
            throw new KnownPlayerException(p);
        }
        //... Normal addPlayer behavior

    } catch(Throwable t) {
        exceptionThrown = true;
        throw t;
    } finally {
        if(!exceptionThrown) {
            // post conditions
            if(isPlayerAccepted(p)) {
                throw new TournamentException(...);
            }
            // invariants
            if(!(getMaxNumPlayers() > 0))
                throw new TournamentException("max players must be greater than 0");
        }
    }
}

```

Mapping to database

- UML object models can be mapped to relational databases:
- Some degradation occurs because all UML constructs must be mapped to a single relational database construct - the table.
- UML mappings
 - Each class is mapped to a table
 - Each class attribute is mapped onto a column in the table
 - An instance of a class represents a row in the table
 - A many-to-many association is mapped into its own table
 - A one-to-many association is implemented as buried foreign key
- Methods are not mapped



User table

id:long	firstName:text[25]	login:text[8]	email:text[32]



LeagueOwner table

id	name	maxNumLeagues	...
56	zoe	12	

Player table

id	name	credits	...
79	john	126	

Database concepts

- Not in this course but already covered in detail in Database Systems
- You should know database concepts of
 - Primary keys, foreign keys
 - Associations/multiplicity
 - Normalization
 - Redundancy



The background of the slide is a photograph of a sunset or sunrise. The sky is filled with dark, textured clouds, with a band of orange and yellow light near the horizon. In the foreground, the dark silhouettes of buildings and trees are visible against the bright horizon.

Case study

UML SCENARIO EXAMPLE

Scenario – library system

- Books and journals

The library contains books and journals. It may have several copies of a given book. Some of the books are for short term loans only. All other books may be borrowed by any library member for three weeks. Members of the library can normally borrow up to six items at a time, but members of staff may borrow up to 12 items at one time. Only members of staff may borrow journals.

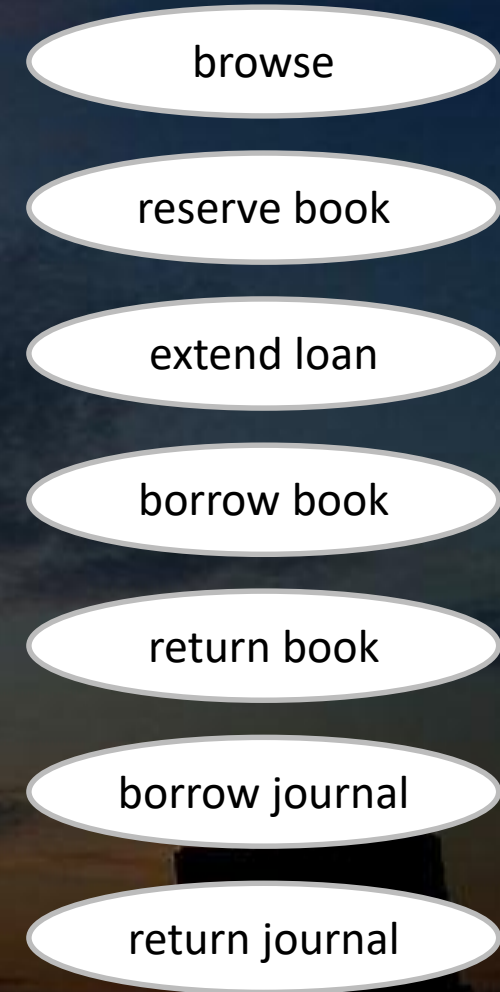
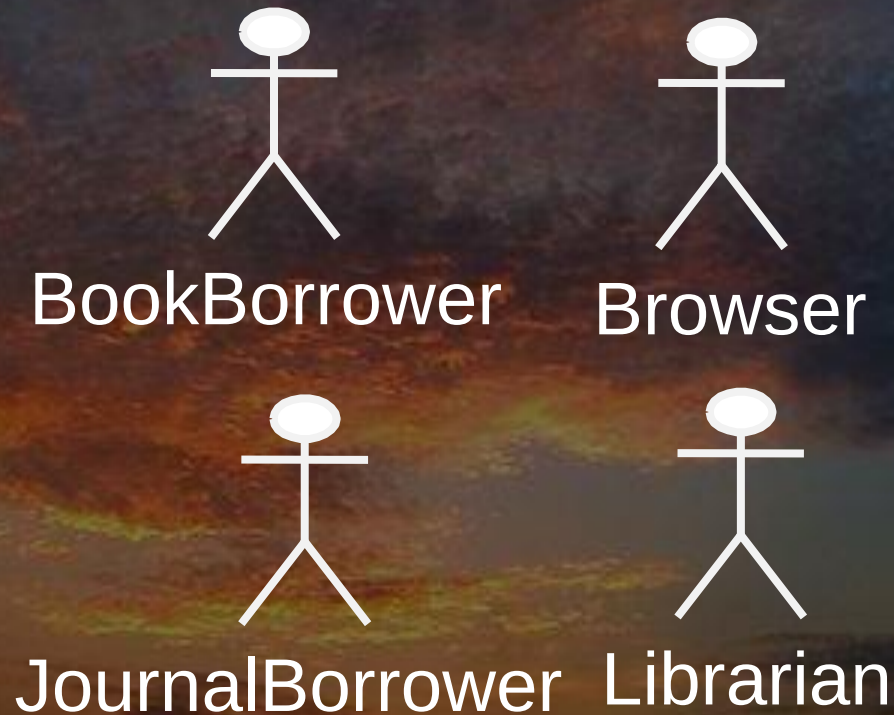
- Borrowing

The system must keep track of when books and journals are borrowed and returned, enforcing the rules described above.

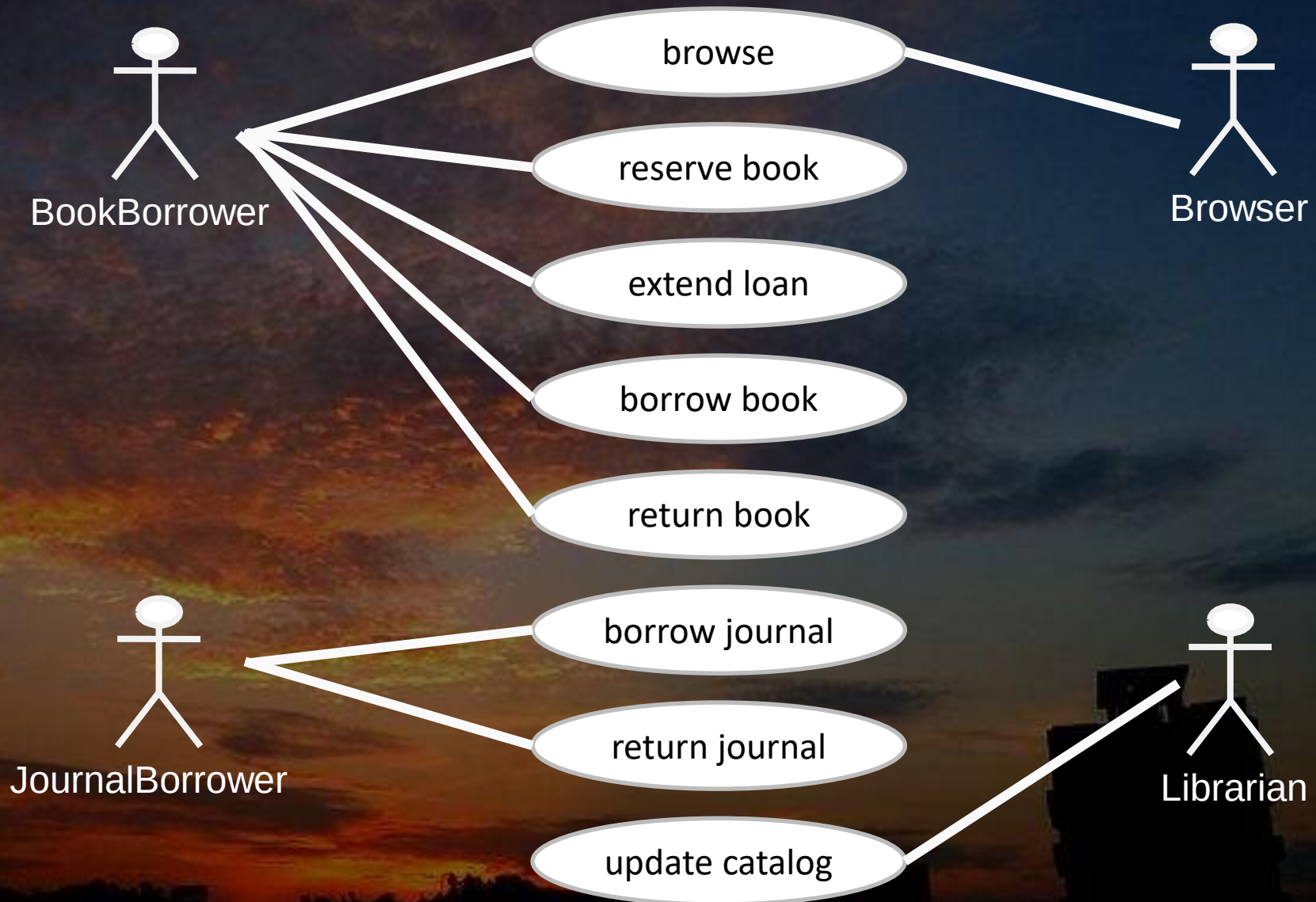
- Browsing

The system should allow users to search for a book on a particular topic, by a particular author etc., to check whether a copy of book is available for loan and, if not, to reserve the book. Anybody can browse in the library.

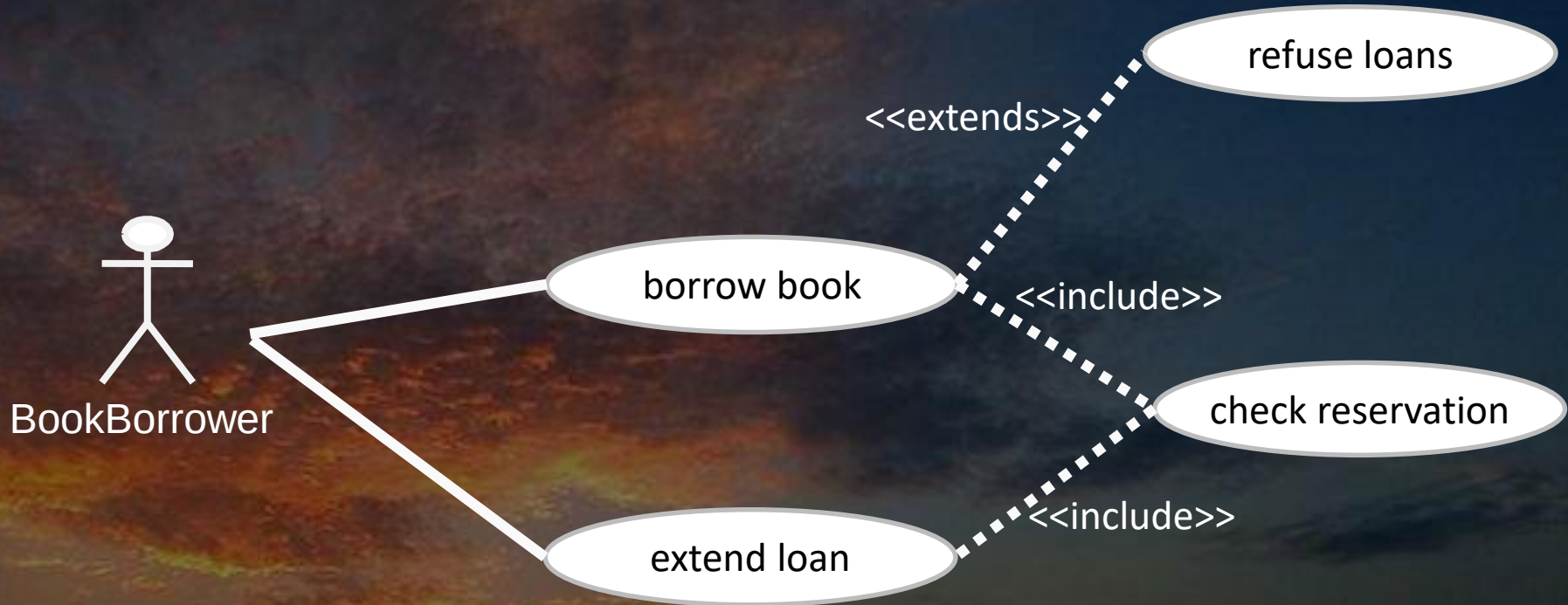
Library system actors & use cases



Library system UCD



Enhancement to UCD





borrow book

status validation:after confirming identity

refuse loans

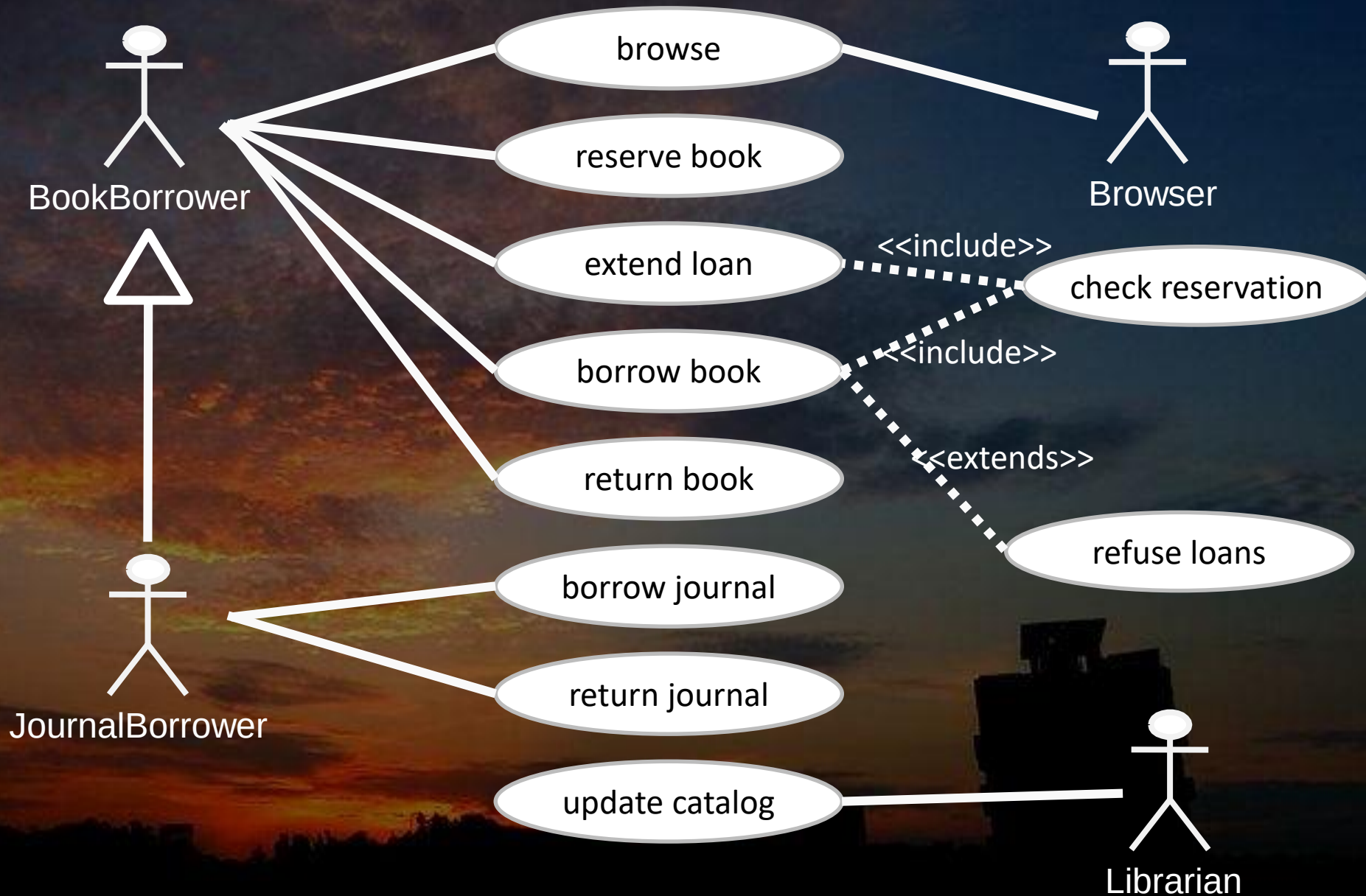
<<extends>>

exceeded maximum books allowed



- Every JournalBorrower is also a BookBorrower
- This indicates a is-a type of relationship → inheritance/generalization

Final UCD



Class diagram

- Take a list of candidate classes by underlining the nouns and noun phrases.
- Books and journals
The library contains books and journals. It may have several copies of a given book. Some of the books are for short term loans only. All other books may be borrowed by any library member for three weeks. Members of the library can normally borrow up to six items at a time, but members of staff may borrow up to 12 items at one time. Only members of staff may borrow journals.
- Borrowing
The system must keep track of when books and journals are borrowed and returned, enforcing the rules described above.

Class diagram

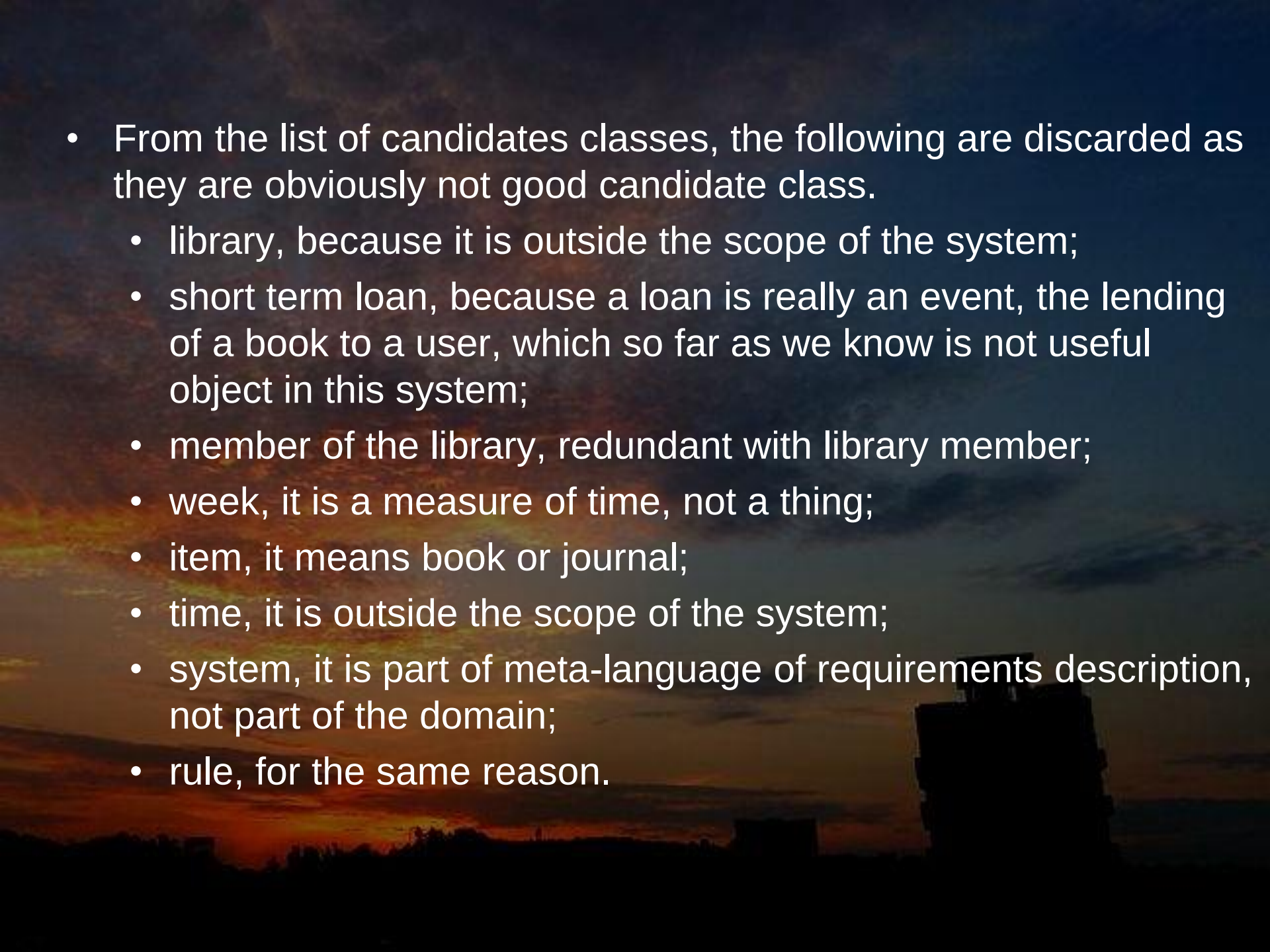
- Take a list of candidates classes by underlining the nouns and noun phrases.

- Books and journals

The library contains books and journals. It may have several copies of a given book. Some of the books are for short term loans only. All other books may be borrowed by any library member for three weeks. Members of the library can normally borrow up to six items at a time, but members of staff may borrow up to 12 items at one time. Only members of staff may borrow journals.

- Borrowing

The system must keep track of when books and journals are borrowed and returned, enforcing the rules described above.

- 
- The background of the slide is a photograph of a sunset or sunrise. The sky is filled with soft, colorful clouds in shades of orange, yellow, and blue. In the lower right corner, there is a dark silhouette of a building, possibly a lighthouse or a tower, which adds a sense of depth and context to the slide.
- From the list of candidates classes, the following are discarded as they are obviously not good candidate class.
 - library, because it is outside the scope of the system;
 - short term loan, because a loan is really an event, the lending of a book to a user, which so far as we know is not useful object in this system;
 - member of the library, redundant with library member;
 - week, it is a measure of time, not a thing;
 - item, it means book or journal;
 - time, it is outside the scope of the system;
 - system, it is part of meta-language of requirements description, not part of the domain;
 - rule, for the same reason.

Possible classes

LibraryMember

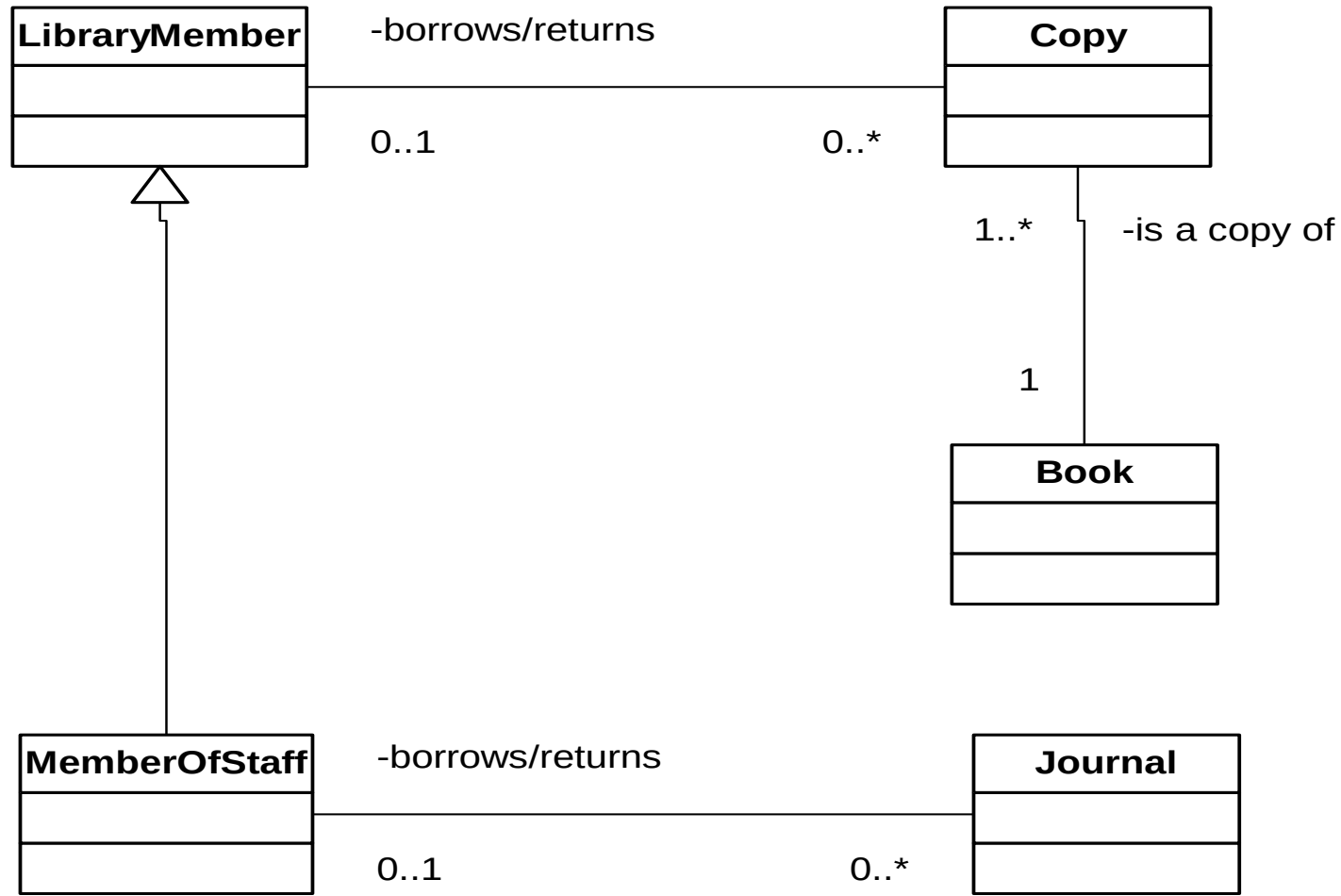
Copy

MemberOfStaff

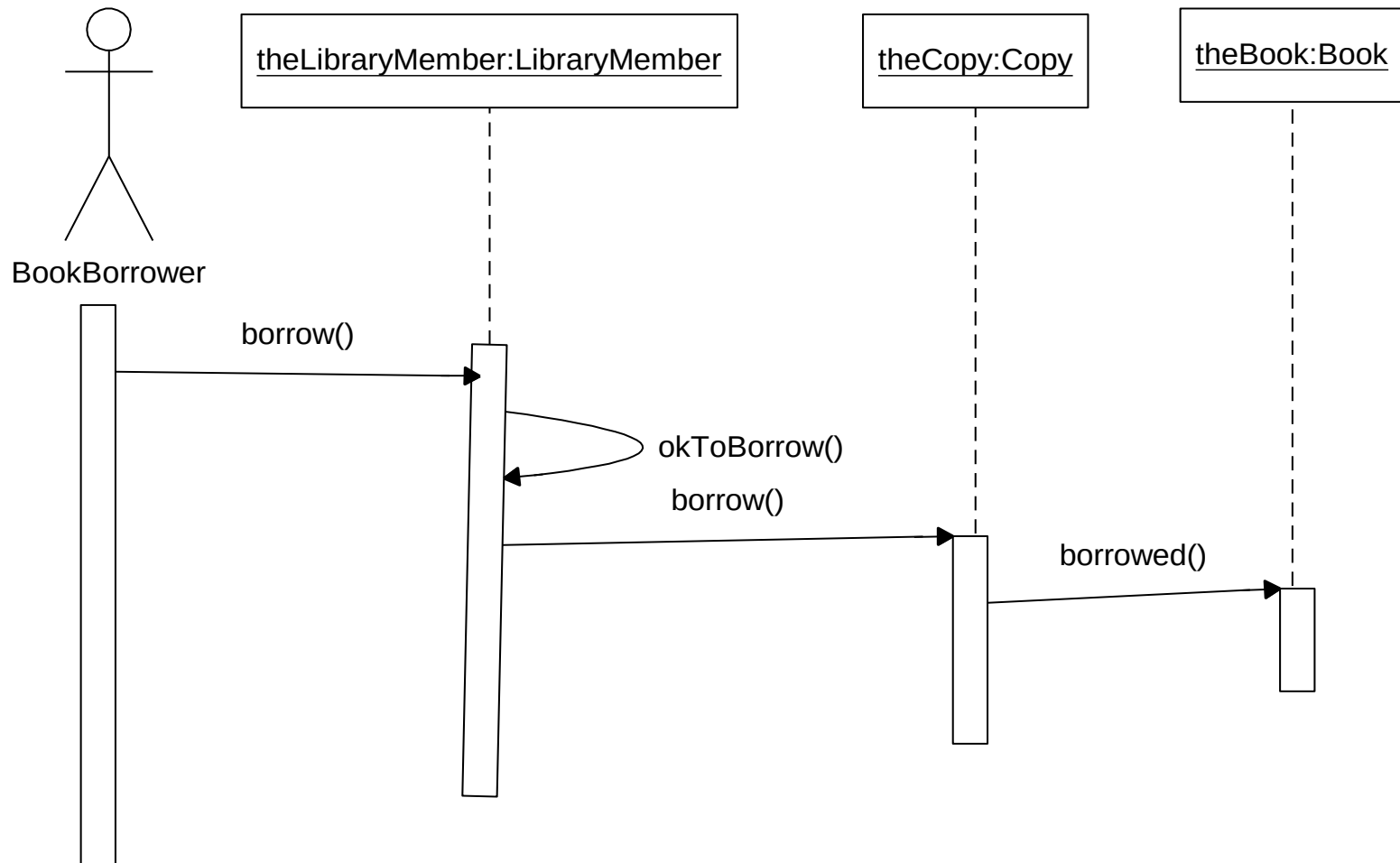
Book

Journal

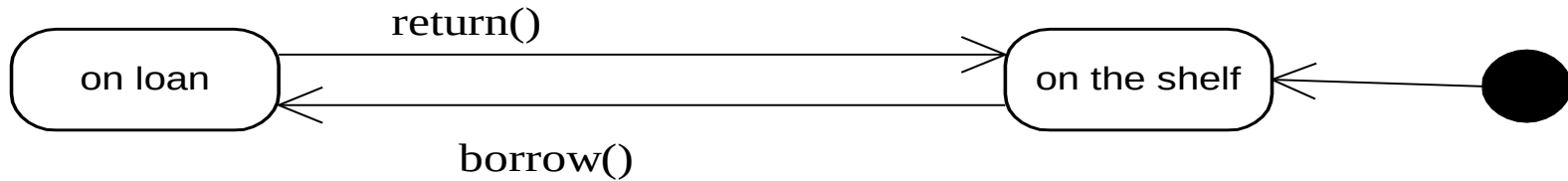
Library system class diagram



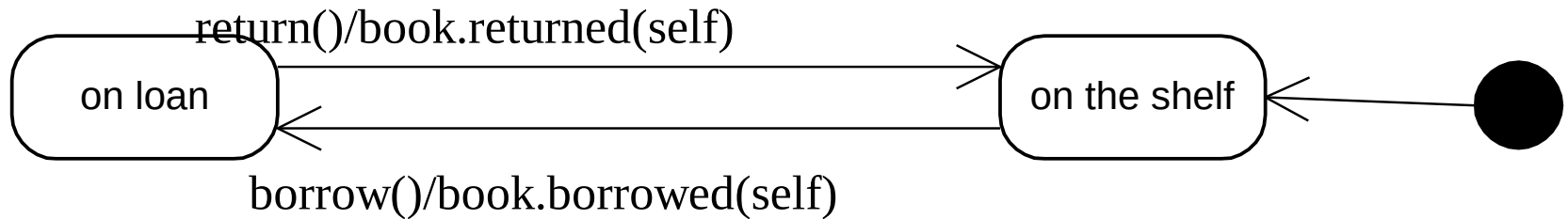
Library system sequence diagram



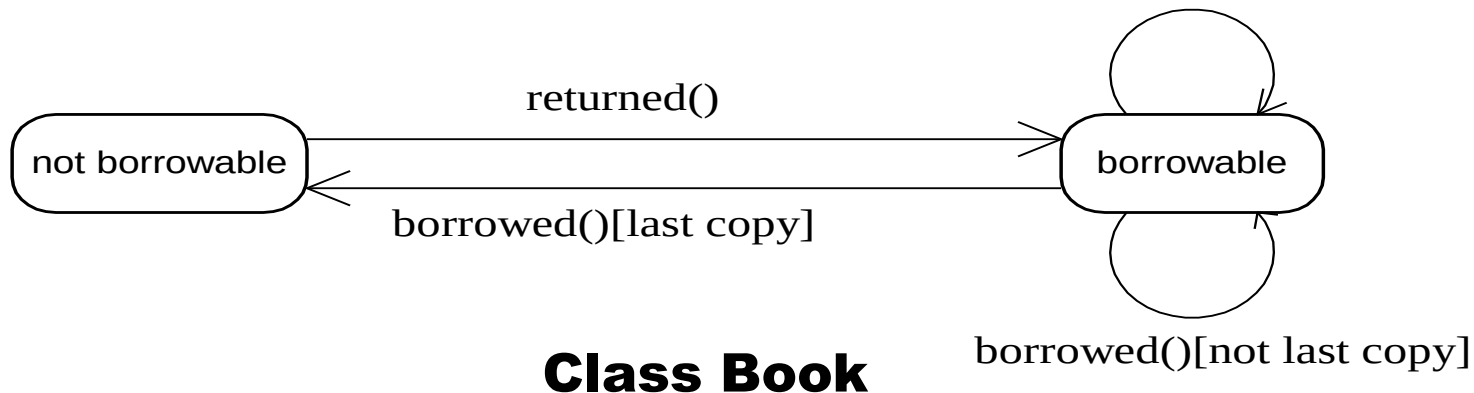
Library system state diagram



Class Copy

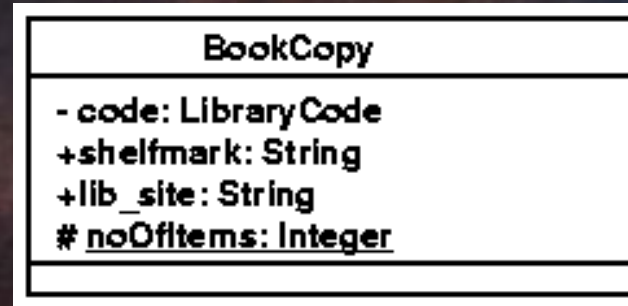


Class Copy with action



Class Book

UML class to Java



```
public class BookCopy {  
    private LibraryCode code;  
    public String shelfmark;  
    public String lib_site;  
    protected static Integer noOfItems;  
    BookCopy() { } //Constructor  
}
```




```
public class Publisher {  
    public Book book[];  
    Publisher() { } //Constructor  
}
```

```
public class Book {  
    private Publisher publisher;  
    Book() { } //Constructor  
}
```



```
public class BookCopy extends BorrowableItem {  
    ...  
    public final void estimatePenalty() { }  
}  
public class Proceedings extends BorrowableItem {  
    ...  
    public final void estimatePenalty() { }  
}  
public abstract class BorrowableItem {  
    ...  
    public abstract void estimatePenalty()  
}
```



```
public abstract class BorrowableItem {  
    ...  
    public synchronized reserve(Date until) {  
    }  
    public static Integer noOfItems() {}  
}
```


Summary

- Four mapping concepts:
 - Model transformation
 - Forward engineering
 - Refactoring
 - Reverse engineering
- Model transformation and forward engineering techniques:
 - Optimizing the class model
 - Mapping associations to collections
 - Mapping contracts to exceptions
 - Mapping class model to storage schemas